

Supplementary materials for: A solution to the learning dilemma for recurrent networks of spiking neurons

Guillaume Bellec^{1,◦}, Franz Scherr^{1,◦}, Anand Subramoney¹,

Elias Hajek¹, Darjan Salaj¹, Robert Legenstein¹, Wolfgang Maass^{1,*}

¹Institute of Theoretical Computer Science, Graz University of Technology,
Inffeldgasse 16b, Graz, Austria

◦ Equal contributions.

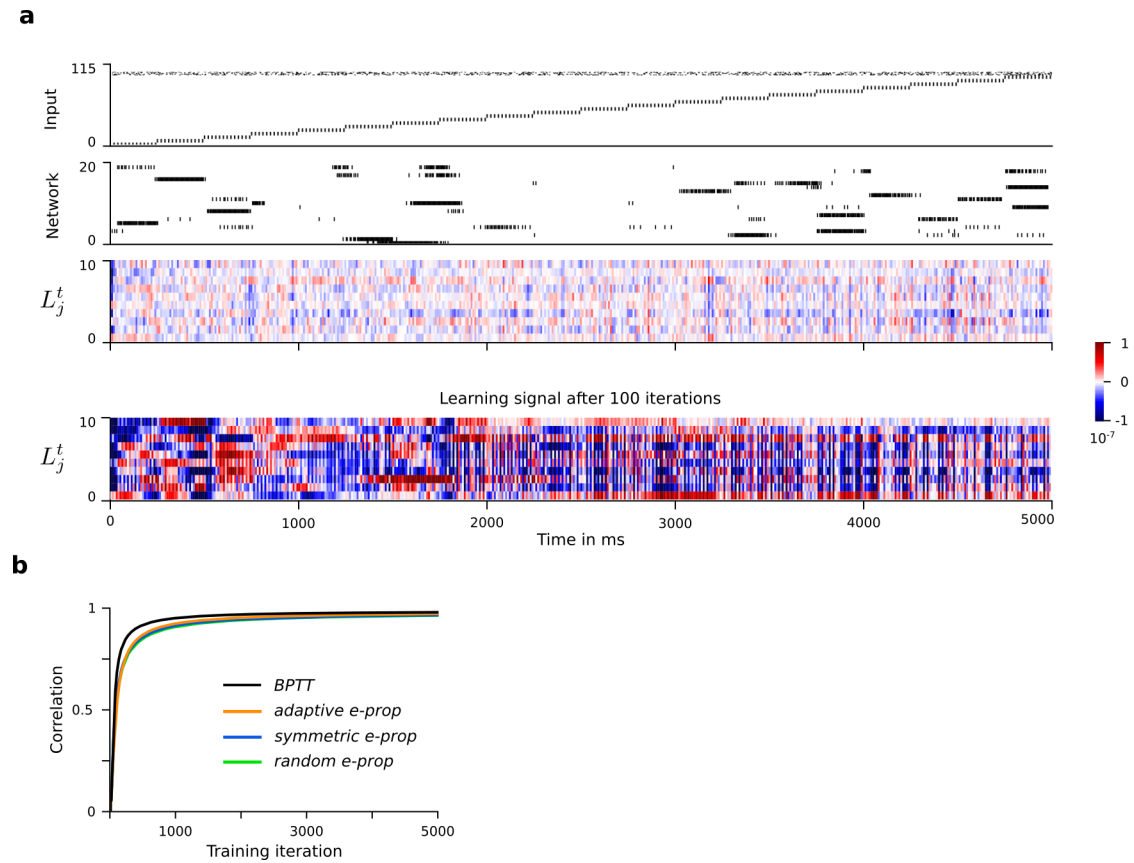
* To whom correspondence should be addressed; E-mail: maass@igi.tugraz.at.

Contents

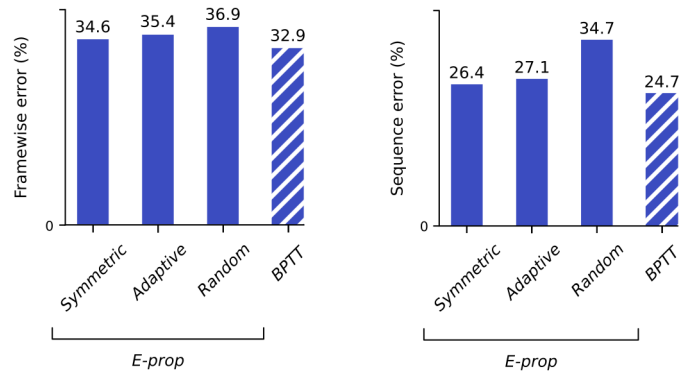
Supplementary Figures	3
Figure 1 Performance comparison of BPTT and e-prop on an extension of the episodic memory task from [1]	3
Figure 2 Comparison of learning algorithms for training LSNNs on the TIMIT task	4
Figure 3 Performance of e-prop on the framewise TIMIT task for LSNNs without recurrent connections	5
Figure 4 LSTM networks trained with BPTT and e-prop on the TIMIT task	6
Figure 5 Performance of reward-based random e-prop and BPTT for the RL version of the task from Figure 3	7
Figure 6 Impact of the length of the simulation time step on the learning performance of e-prop for the task of Figure 3	8
Figure 7 Evaluation of several variants of random e-prop	8
Figure 8 Test of a prediction of our analysis when e-prop is likely to approximate the performance of BPTT, and when not	10
Figure 9 Histogram of membrane potentials	11
Supplementary Tables	12
Table 1 Table of hyperparameters used in Atari tasks	12
Supplementary Notes	13
1 Eligibility traces	13
Eligibility traces for LSNNs with membrane potential reset	13
Eligibility traces for LSNNs with non-uniform synaptic delays	14
2 Optimization and regularization procedures	14
Optimization procedure	14
Firing rate regularization for LSNNs	15
Adaptive e-prop and weight decay regularization	15
Voltage regularization	15
Optimization with rewiring for sparse network connectivity	16
3 Supervised learning with e-prop	17
Synaptic plasticity rules for e-prop in supervised learning	17
Simulation details: framewise phoneme classification (Figure 2)	18
Simulation details: phoneme sequence recognition with CTC (Figure 2)	19

Applying e-prop to an episodic memory task	20
Simulation details: task where temporal credit assignment is difficult (Figure 3)	21
4 Applying supervised learning with e-prop to artificial neural networks (LSTM networks)	22
Phoneme recognition with LSTM networks and e-prop	22
Simulation details: phoneme recognition with LSTM networks (Supplementary Figure 4)	23
Simulation details: phoneme sequence recognition with CTC and LSTM (Supplementary Figure 4)	23
LSTM network model	23
Eligibility traces for LSTM units	24
5 Reward-based e-prop: Application of e-prop to deep RL	25
Synaptic plasticity rules for reward-based e-prop	25
Simulation details: RL version of the task where credit assignment is difficult (Supplementary Figure 5)	28
Simulation details: Atari task (Figures 4 and 5)	28
6 Evaluation of four variations of <i>e-prop</i> (Supplementary Figure 7)	30
truncated eligibility trace for LIF neurons	30
Global broadcast weights	30
Temporally local broadcast weights	30
Replacing the eligibility trace by the corresponding term of the Clopath rule	31
Simulation details: pattern generation task	31

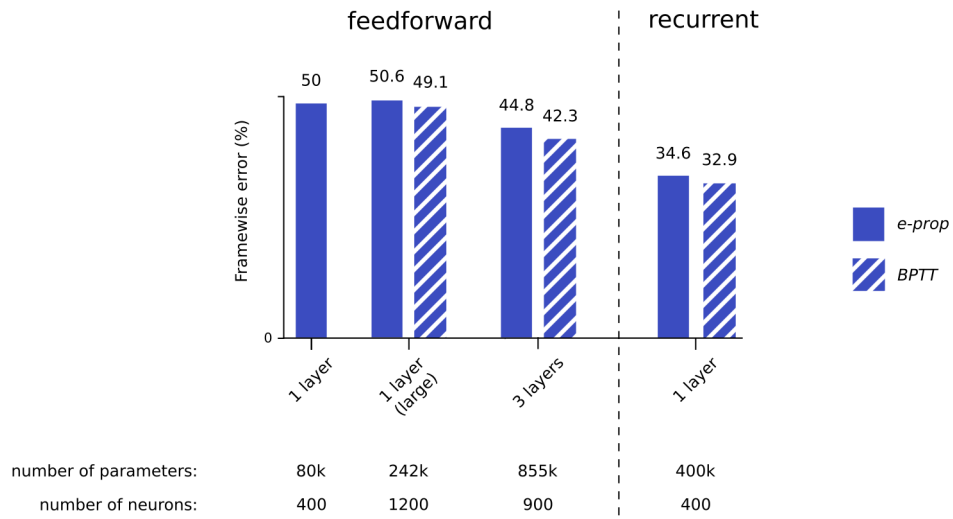
Supplementary Figures



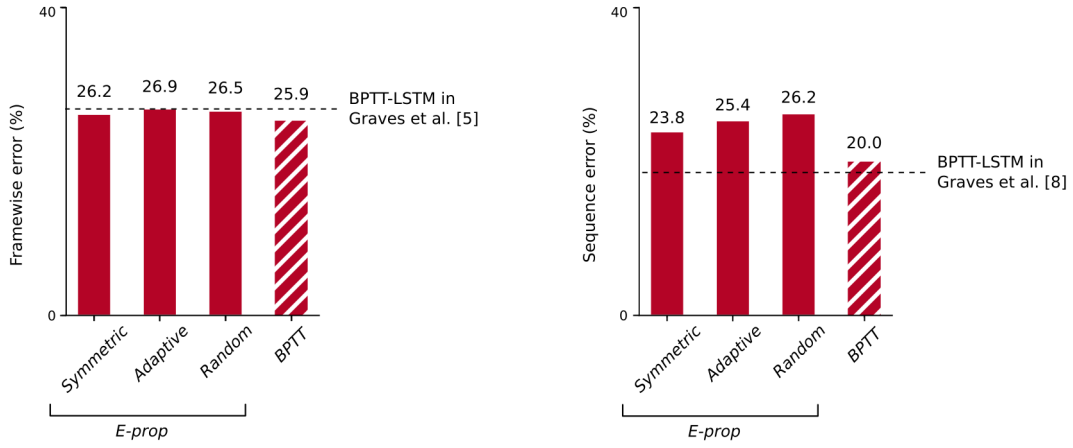
Supplementary Figure 1: **Performance comparison of BPTT and e-prop on an extension of the episodic memory task from [1]**. The LSNN had to generate at each time step the values of all pixels that described the video frame of the movie at that time step, see section 3. **a)** Input spikes, network activity (for 20 sample neurons), learning signals, and network outputs (at 1s and 4s, shown at the top) of an LSNN after 1000 training iterations. For comparison we also show learning signals after just 100 iterations, where their amplitude is still large. **b)** Performance of BPTT and e-prop.



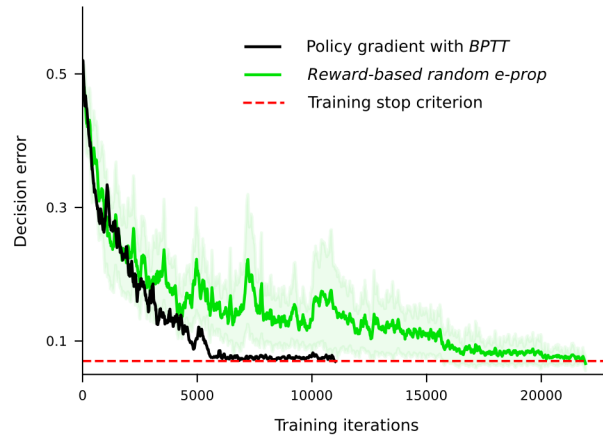
Supplementary Figure 2: **Comparison of learning algorithms for training LSNNs on the TIMIT task.** Performance of BPTT and the three versions of e-prop on frame-wise phoneme classification (left) and for phoneme sequence recognition (right).



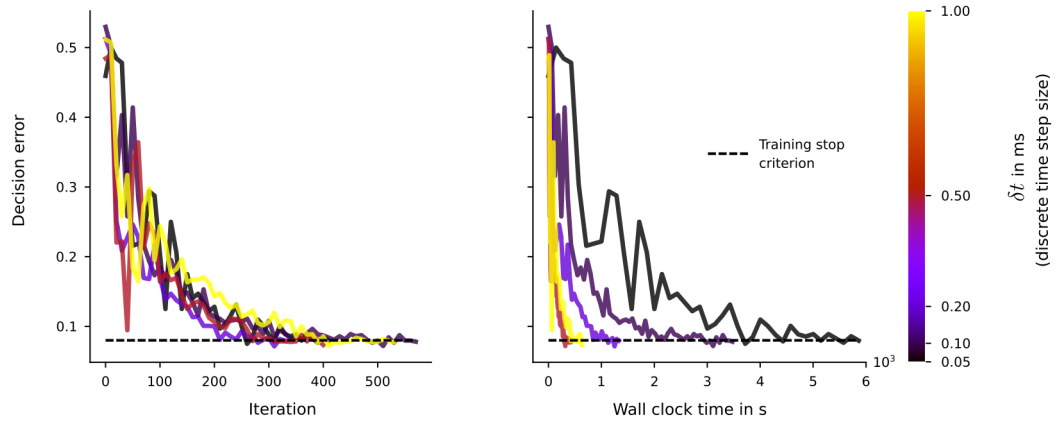
Supplementary Figure 3: **Performance of e-prop on the framewise TIMIT task for variations of the LSNN from Figure 2 without recurrent connections (left of dashed line).** The result of the left-most bar was achieved with the same hyperparameters as used in Figure 2. For the other feedforward architectures we modified hyperparameters to optimize the performance. The results of Figure 2 c) for LSNNs with recurrent connections are redrawn on the right of the dashed line. One sees that recurrent connections are essential for achieving good performance on this task.



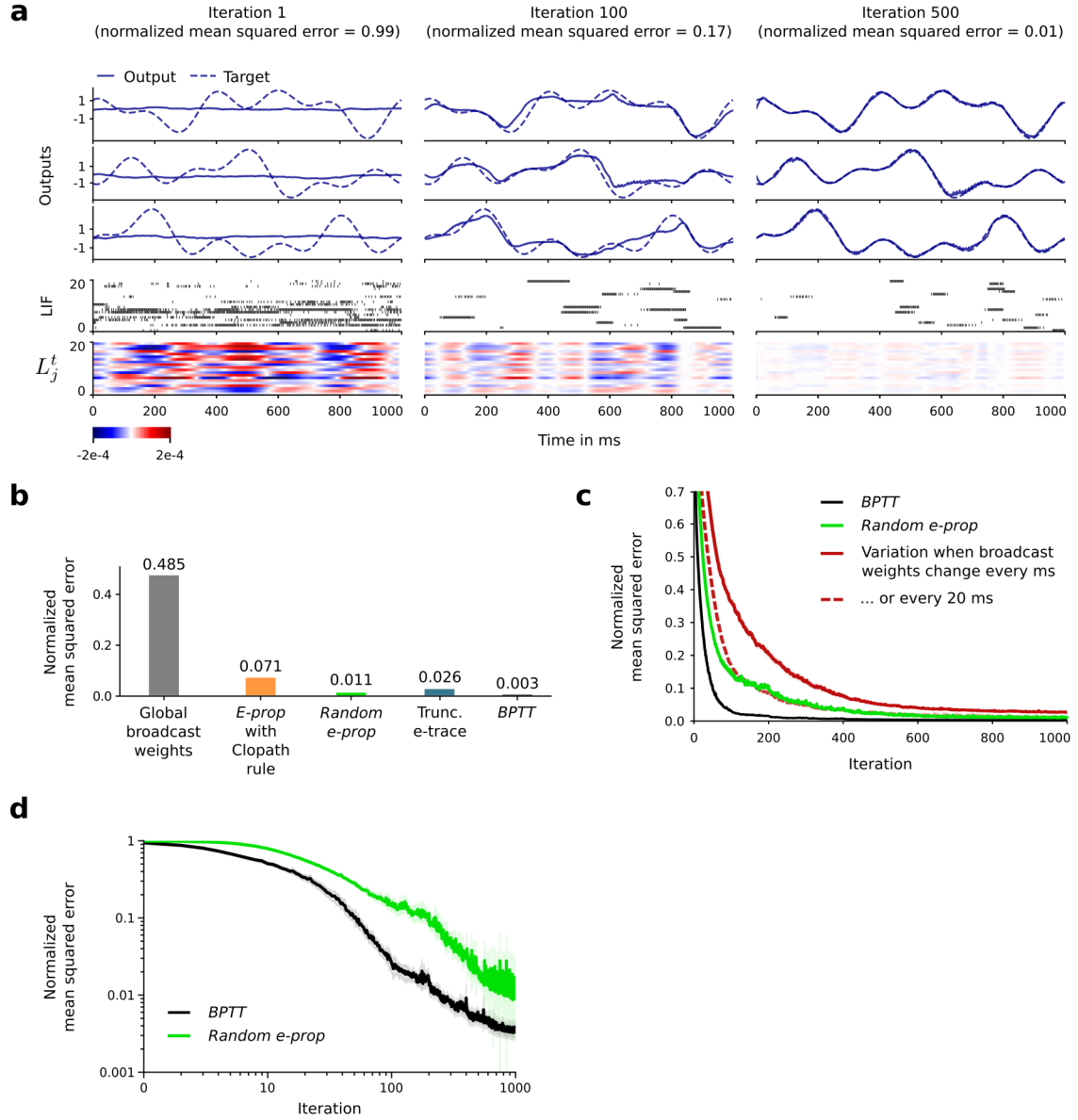
Supplementary Figure 4: **LSTM networks trained with BPTT and e-prop on the TIMIT task.** Performance of BPTT and the three versions of e-prop on frame-wise phoneme classification (left) and for phoneme sequence recognition (right). One sees that e-prop approximates BPTT performance also for non-spiking neural networks. The BPTT baselines aim at reproducing the results obtained with LSTM networks in [2] and [3]. The network architectures and audio pre-processing settings are taken from [2, 4] for framewise phoneme classification and from [3] for phoneme sequence transcription. In comparison to the BPTT-LSTM baselines that we could achieve in this way, 26.9% framewise error rate was reported in [2] and 18.6% sequence error rate was reported in [3].



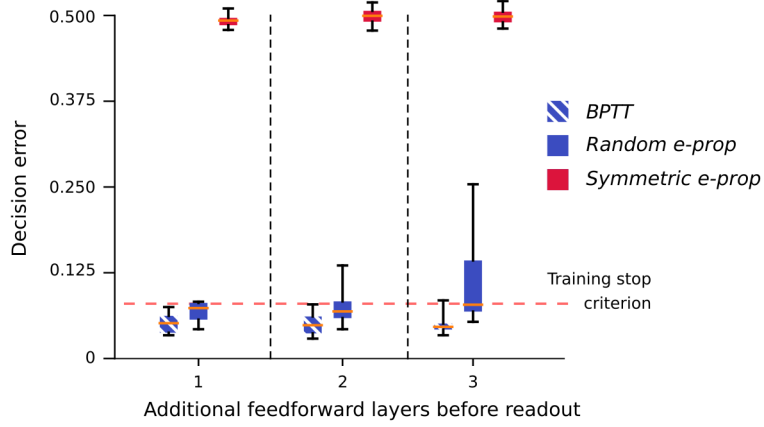
Supplementary Figure 5: **Performance of reward-based random e-prop and BPTT for the RL version of the task from Figure 3, applied to an LSNN consisting of 50 LIF and 50 ALIF neurons.**



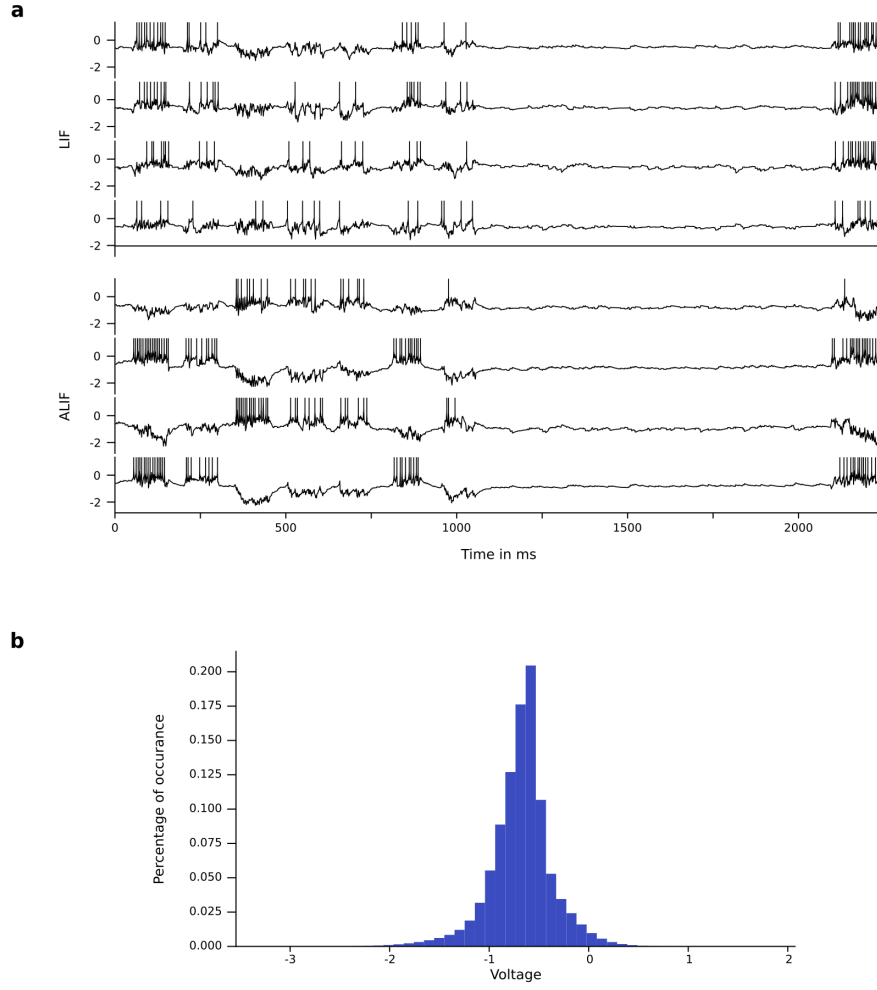
Supplementary Figure 6: **Impact of the length of the simulation time step on the learning performance of e-prop for the task of Figure 3 (simplified version with 5 cues instead of 7).** Decision error for this task is shown as function of the number of training iterations, and as function of wall clock time. The results are averaged over 5 different seeds. One sees that the length of the simulation time step has no visible impact on the learning performance of e-prop. However, smaller simulation time steps lead to substantially larger simulation time



Supplementary Figure 7: **Evaluation of several variants of random e-prop** **a)** The task is a classical benchmark task for learning in recurrent SNNs: learning to generate a target pattern, extended here to the challenge to simultaneously learn to generate 3 different patterns, which makes credit assignment for errors more difficult. Learning performance with random e-prop is shown after training for 1, 100, 500 s. **b)** Normalized mean squared error of several learning algorithms for this task after 500 s of training. “Clopath rule” denotes a replacement of the eligibility trace of random e-prop by a corresponding term proposed in [5] based on experimental data. **c)** Learning curves for variations of random e-prop with temporally local broadcast weights. Results were averaged over 20 runs. **d)** Comparison of learning curves for BPTT and random e-prop with logarithmic scale.



Supplementary Figure 8: **Test of a prediction of our analysis when e-prop is likely to approximate the performance of BPTT, and when not.** We blocked in the learning task of Figure 3 route i for gradient information by inserting 1 to 3 feedforward layers (completely connected from one layer to the next) between the RSNN and the readouts y of Figure 2a. All neurons in the RSNN were connected to the first feedforward layer, and all neurons in the last layer to the readout neurons. The decision error is reported for a distribution of runs with different initialization weights. The upper and lower whiskers are the minimum and the maximum, the orange bar is the median and the box extends from the lower to the upper quartile of the data. As predicted, the feedforward layers drastically reduce the performance of symmetric e-prop, but not of BPTT. Thus for successful training of a RSNN by symmetric e-prop it is essential that all or most neurons in the RSNN have a synaptic connection to readout neurons. Random e-prop works still quite well, but the high variance of its performance for 3 intermediate feedforward layers suggests some learning instability. See section 3 for details of this experiment.



Supplementary Figure 9: **Further details on membrane potentials of neurons in the network of Figure 3 a.** Traces of voltage – firing threshold ($v_j^t - A_j^t$) for 4 randomly selected LIF and 4 ALIF neurons. **b)** Histogram of these values for all neurons in the network and all time points t .

Supplementary Tables

	Pong	Fishing Derby
τ_m	15 ms	10 ms
v_{th}	1	1
τ_a	70 ms	70 ms
β	0.1	0.1
τ_{out}	2 ms	4 ms
$v_{th,CNN}$	0.1	0.1
$\tau_{m,CNN}$	1 ms	1 ms
c_V	1	0.5
c_H	0.0025	0.006
c_{reg}	50	10
$c_{reg,V}$	10^{-4}	10^{-3}
ΔT	$\left(\left(\sqrt[3]{\frac{T_{max}}{1\ s} \cdot 1000} + 1 \right)^3 - \frac{T_{max}}{1\ s} \cdot 1000 \right) \text{ ms}$	1000 ms
First convolutional Layer		
Kernel structure	$8 \times 8 \times 1 \times 8$	$8 \times 8 \times 1 \times 16$
Stride	4	4
Padding	VALID	SAME
Number of neurons	3200	7056
Second convolutional Layer		
Kernel structure	$4 \times 4 \times 8 \times 8$	$4 \times 4 \times 16 \times 32$
Stride	1	2
Padding	VALID	SAME
Number of neurons	2312	3872

Supplementary Table 1: Table of hyperparameters used in Atari tasks

Supplementary Notes

1 Eligibility traces

Eligibility traces have been introduced in section “Mathematical basis for e-prop” in Results. Here, we provide further information on eligibility traces. First, we discuss an alternative view on eligibility traces as derivatives. Second, we extend our treatment of eligibility traces for LSNNs in Methods to include non-uniform synaptic delays.

Viewing eligibility traces as derivatives

The notion of derivative $\left[\frac{d\mathbf{h}_j^t}{dW_{ji}}\right]_{\text{local}}$ from Equation (14) in Methods quantifies the influence of an infinitesimal change of W_{ji} on the hidden state $\mathbf{h}_j^t$ through the internal processes of neuron j and the spikes of neuron i . Unlike the partial derivative $\frac{\partial \mathbf{h}_j^t}{\partial W_{ji}}$ it does not only take into account the update of the hidden state at time step t , but considers instead the full neuron history. In comparison to the total derivative $\frac{d\mathbf{h}_j^t}{dW_{ji}}$ it is only aware of the activity of neurons i and j . Below we provide a formal explanation why the eligibility traces and the eligibility vectors can be viewed as derivatives.

One also arrives at the Equation (13) in Methods if one computes the total derivative $\frac{d\mathbf{h}_j^t}{dW_{ji}}$ in a variation of the computational graph of Figure 6 where the function $\mathbf{h}_j^t = M(\mathbf{h}_j^{t-1}, \tilde{\mathbf{z}}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$ receives a vector $\tilde{\mathbf{z}}^{t-1}$ that is equal to $\mathbf{z}^{t-1}$ but considered as a constant (i.e. the gradients are stopped at this node in the computational graph). Hence one can define the notation $\left[\frac{dA}{dB}\right]_{\text{local}}$ as a total derivative $\frac{dA}{dB}$ in this altered computational graph. To prove this equivalence one follows a derivation similar to the one detailed after equation (15) in Methods but in the altered computational graph: the total derivative $\left[\frac{d\mathbf{h}_j^t}{dW_{ji}}\right]_{\text{local}}$ expands as $\sum_{t \leq t'} \left[\frac{d\mathbf{h}_j^{t'}}{d\mathbf{h}_j^t}\right]_{\text{local}} \frac{\partial \mathbf{h}_j^{t'}}{\partial W_{ji}}$. Viewing that $\left[\frac{d\mathbf{h}_j^t}{d\mathbf{h}_j^{t-1}}\right]_{\text{local}} = \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}}$ in this computational graph, one recognizes that $\left[\frac{d\mathbf{h}_j^t}{dW_{ji}}\right]_{\text{local}}$ is the eligibility vector given in Methods, Equation (14). Equation (13) follows since $e_{ji}^t = \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \cdot \epsilon_{ji}^t = \left[\frac{dz_j^t}{dW_{ji}}\right]_{\text{local}}$. In fact this definition allows the extension of this new local derivative to other quantities and one can summarize symmetric e-prop as the replacement of $\frac{dE}{dW_{ji}}$ by $\left[\frac{dE}{dW_{ji}}\right]_{\text{local}}$ in stochastic gradient descent.

Eligibility traces for LSNNs with membrane potential reset

The eligibility traces derived in the methods do not take the reset term into account. We derive here the eligibility traces that can correct for this. Note however that we did not observe an improvement when using this more complex model on the phoneme recognition and the task where temporal credit assignment is difficult.

Eligibility traces for LIF neurons: When taking into account the reset, the partial derivative $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t}$ becomes $\alpha - v_{\text{thr}} \psi_j^t$ instead of α and, accordingly to Equation (14) from Methods, the eligibility vector can be computed with the recursive formula: $\epsilon_{ji}^{t+1} = (\alpha - \beta \psi_j^t) \epsilon_{ji}^t + z_j^t$.

Eligibility traces for ALIF neurons: According to the dynamics of the ALIF neurons defined in Equations (6)–(10) in Methods one coefficient differs in the matrix $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t} \in \mathbb{R}^{2 \times 2}$ as soon as one takes the reset into account. The coefficient $\frac{\partial v_j^t}{\partial a_j^t}$ was 0 without reset and becomes now $v_{\text{thr}} \beta \psi_j^t$.

Overall the full derivative $\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t}$ is then equal to:

$$\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t} = \begin{pmatrix} \alpha - v_{\text{thr}} \psi_j^t & v_{\text{thr}} \beta \psi_j^t \\ \psi_j^t & \rho - \beta \psi_j^t \end{pmatrix}. \quad (1)$$

Even-though e-prop can be implemented as such, the recursive propagation of the eligibility vector in Equation (14) from Methods cannot be written in the form of two separable Equations as done in Equations (24) and (25) in Methods. Hence, we preferred to ignore the reset in Methods to provide more interpretable Equations for eligibility traces.

Eligibility traces for LSNNs with non-uniform synaptic delays

In our derivation of eligibility traces for LSNNs, we used uniform synaptic delays to ease notation. Here, we detail how e-prop can be extended to non-uniform delays of several ms. Let the delay of a synapse from neuron i to j be denoted by $c(j, i) > 0$. Similarly, let $d(j, i) \geq 0$ be the delay of a synapse that connects an input neuron i with neuron j . Using this definition, the dynamics of the membrane potential, see Equation (6) in Methods, is written as:

$$v_j^{t+1} = \alpha v_j^t + \sum_{i \neq j} W_{ji}^{\text{rec}} z_i^{t-c(j,i)} + \sum_i W_{ji}^{\text{in}} x_i^{t-d(j,i)} - z_j^t v_{\text{th}}. \quad (2)$$

Like in the uniform delay case, we obtain $\frac{\partial v_j^{t+1}}{\partial v_j^t} = \alpha$. The difference for arbitrary delays becomes visible in $\frac{\partial v_j^t}{\partial W_{ji}^{\text{rec}}} = z_i^{t-c(j,i)}$ and in $\frac{\partial v_j^t}{\partial W_{ji}^{\text{in}}} = x_i^{t-d(j,i)}$. For recurrent weights, the component of the eligibility vector associated to the membrane potential is hence:

$$\epsilon_{ji,v}^t = \sum_{t' \leq t-c(j,i)} z_i^{t'} = \bar{z}_i^{t-c(j,i)}. \quad (3)$$

As the dynamics of the threshold adaptation is unchanged, the update of $\epsilon_{ji,a}^t$ remains as given in Equation (24) in Methods. We obtain an eligibility trace

$$e_{ji}^t = \psi_j^t \left(\bar{z}_i^{t-c(j,i)} - \beta \epsilon_{ji,a}^t \right). \quad (4)$$

Analogously, we obtain the corresponding eligibility trace for input synapses by replacing z_i^t and $c(j, i)$ with x_i^t and $d(j, i)$ respectively.

2 Optimization and regularization procedures

Here, we discuss how optimization of networks was implemented and techniques that were used to regularize networks.

Optimization procedure

For e-prop and for BPTT, the weights were updated once after a batch of training trials. For simplicity, all the weight updates $\Delta W_{ji}^{\text{rec}}$ are written for the most basic version of stochastic gradient descent ($\Delta W_{ji}^{\text{rec}} = -\eta \widehat{\frac{dE}{dW_{ji}^{\text{rec}}}}$, where $\widehat{\frac{dE}{dW_{ji}^{\text{rec}}}}$ is the gradient estimate) in this article. In practice, we used Adam [6] to boost stochastic gradient descent. We refer to [6] for the computation of the weight updates that result from the gradient estimates.

Firing rate regularization for LSNNs

To ensure a low firing rate in LSNNs, we added a regularization term E_{reg} to the loss function E . This regularization term had the form:

$$E_{\text{reg}} = \frac{1}{2} \sum_j (f_j^{\text{av},t} - f^{\text{target}})^2, \quad (5)$$

where f^{target} is a target firing rate and $f_j^{\text{av}} = \frac{1}{Tn_{\text{trial}}} \sum_t z_j^t$ is the average firing rate of neuron j . Here, the sum runs over the time steps of all the n_{trial} trials between two weight updates. To derive the plasticity rule that implements this regularization, we follow Equation (27) in Methods. The partial derivative of the regularization loss has the form:

$$\frac{\partial E_{\text{reg}}}{\partial z_j^t} = \frac{1}{Tn_{\text{trial}}} (f_j^{\text{av}} - f^{\text{target}}). \quad (6)$$

Inserting this expression into Equation (27) in Methods, we obtain the plasticity rule that implements the regularization:

$$\Delta W_{ji}^{\text{rec}} = \eta c_{\text{reg}} \sum_t \frac{1}{Tn_{\text{trial}}} (f^{\text{target}} - f_j^{\text{av}}) e_{ji}^t, \quad (7)$$

where c_{reg} is a positive coefficient that controls the strength of the regularization. This plasticity rule is applied simultaneously together with the plasticity rule that minimizes the loss E . Note that this weight update fits the e-prop framework provided by Equation (1) from Results with a learning signal L_j^{reg} proportional to $f^{\text{target}} - f_j^{\text{av}}$ available locally at neuron j . The learning signal L_j^{reg} can be computed locally and added to the task-specific learning signal L_j^t . A variant of this regularization replaces the average firing rate f_j^{av} with a running average $f_j^{\text{av},t}$ that accumulates the spikes since the last weight update. It can achieve the same effect with less resources when implemented online.

Adaptive e-prop and weight decay regularization

In adaptive e-prop all neurons are receiving learning signals with random broadcast weights B_{jk} as in random e-prop. Yet, if neuron j sends activity to the output neuron k via output weights W_{kj}^{out} , the broadcast weights B_{jk} are subsequently adapted. This was implemented by applying an identical weight update to the broadcast and the output weights, i.e. $\Delta B_{jk} = \Delta W_{kj}^{\text{out}}$ and by subtracting $c_{\text{decay}} \cdot W_{kj}^{\text{out}}$ (resp. $c_{\text{decay}} \cdot B_{jk}$) from the weight W_{kj}^{out} (resp. B_{jk}) after each weight update, where $c_{\text{decay}} > 0$ is the regularization factor (see specific experiments for the value of c_{decay}). This weight decay in combination with the mirroring of the weight updates has the effect that, despite different initialization, the output weights and the adaptive broadcast weights converge to similar values. The remaining difference of performance between symmetric and adaptive e-prop reported in Supplementary Figure 2 and Supplementary Figure 4 may be explained by the different initializations.

Adaptive e-prop can be viewed as that version of e-prop that exploits all types of biologically plausible learning signals that are available at individual neurons, apart from reward prediction errors or signals from a separately trained RSSN that sends learning signals, see Figure 3 of [7].

Voltage regularization

One advantage of a gradient-based learning method is that additional terms can be added to the loss function in order to enforce a particular network behavior. We utilized this possibility to regularize the membrane voltage of network neurons, using an idea of Arjun Rao. This regularization reduced excessive membrane voltages (which can otherwise even exceed the threshold voltage during the neuron’s refractory period) and led to sparser and more irregular firing activity, similar

as in biological data. In particular, we used a regularization term that penalized membrane potentials higher than the threshold voltage A_t for ALIF neurons (resp. v_{thr} for LIF) or lower than $-A_t$ for ALIF neurons (resp. $-v_{\text{thr}}$ for LIF). We add a regularization loss of the form:

$$E_{V,\text{reg}} = E_{\text{reg},V+} + E_{\text{reg},V-} = \frac{1}{2} \sum_t \left([v_j^t - A_j^t]^+ \right)^2 + \left([-v_j^t - A_j^t]^+ \right)^2 \quad (8)$$

for ALIF neurons (replace A_t by v_{thr} for LIF neurons), where $[\cdot]^+$ is the rectified linear function. Unlike other loss functions, $E_{V,\text{reg}}$ depends directly on the voltage and not on the spikes. Hence instead of the full e-prop formula we use that the cost function is already local to neuron j , so that we can derive the weight update as follows (we consider only the first part $E_{\text{reg},V+}$ of the loss function to simplify the analysis):

$$\Delta W_{ji} = -\eta c_{\text{reg},V} \sum_t \left[\frac{dE_{\text{reg},V+}}{dW_{ji}} \right]_{\text{local}} \quad (9)$$

$$= -\eta c_{\text{reg},V} \sum_t \frac{\partial E_{\text{reg},V+}}{\partial \mathbf{h}_j^t} \epsilon_j^t \quad (10)$$

$$= -\eta c_{\text{reg},V} \sum_t [v_j^t - A_j^t]^+ (\epsilon_{ji,v}^t - \epsilon_{ji,a}^t) \quad (11)$$

$$= -\eta c_{\text{reg},V} \sum_t [v_j^t - A_j^t]^+ (\bar{z}_i^{t-1} - \epsilon_{ji,a}^t) . \quad (12)$$

Optimization with rewiring for sparse network connectivity

Due to limited resources, neural networks in the brain and in neuromorphic hardware are sparsely connected. In addition, the connectivity structure of brain networks is dynamic, with synaptic connections being added and deleted on the time scale of hours or days, which was shown to help the network to use the limited connectivity resources in an optimal manner [8]. In order to test whether e-prop is compatible with synaptic rewiring, we combined it with DEEP R [9]. DEEP R is based on a model for synaptic rewiring in the brain [8] and allows to rewire sparse neural network models during training with gradients descent. The algorithm minimizes the loss function E subject to a constraint on the total number of connected synapses. To do so, each synaptic weight W_{ji} is assigned a fixed sign s_{ji} (it is defined to be excitatory or inhibitory) and an amplitude w_{ji} . Each potential synaptic connection can either be “active”, i.e., the synaptic connection is realized, or “dormant”, i.e., this potential connection is not realized.

For a dormant synaptic connection, the weight W_{ji} is set to be zero and the gradients and weight updates of the connection $i \rightarrow j$ are not computed. It means in e-prop that dormant synapses do not require eligibility traces. For an active connection, the weight is defined as $W_{ji} = s_{ji} w_{ji}$ and the weight amplitude is updated according to the update $\Delta w_{ji} = s_{ji} \Delta W_{ji} - \eta c_{L1}$ where ΔW_{ji} is the weight update given here by e-prop and $c_{L1} = 0.01$ is an $L1$ regularization coefficient. To update the network structure such that the set of active connections is optimized along side their synaptic weights, DEEP R proceeds as follows after each weight update:

- every active connection for which the amplitude becomes negative is set to be dormant,
- and some dormant connections are selected randomly and set to be active with $w_{ji} = 0$ such that the total number of active connection remains constant.

We define the synapse signs s_{ji} such that 80% of the neurons are excitatory and 20% are inhibitory. Despite the constraint on the neuron signs and the constraint that 90% of the synapses should remain dormant throughout the learning process, e-prop and rewiring solve the task where temporal credit assignment is difficult of Figure 3.

3 Supervised learning with e-prop

Synaptic plasticity rules for e-prop in supervised learning

Here, we derive synaptic plasticity rules that result from e-prop for supervised learning. We consider two cases: First, we derive plasticity rules for regression tasks, and second, for classification tasks.

We follow the scheme described by Equation (27) in Methods. Hence the loss gradients $\frac{dE}{dW_{ji}}$ are estimated using the approximation $\widehat{\frac{dE}{dW_{ji}}} \stackrel{\text{def}}{=} \sum_t \frac{\partial E}{\partial z_j^t} e_{ji}^t$. Given the eligibility traces that are derived in Methods for spiking neurons and Supplementary Note 4 for LSTM, what remains to be derived for each task is the expression of the relevant derivative $\frac{\partial E}{\partial z_j^t}$ and show that it can be computed online.

Regression tasks: Consider a regression problem with loss function $E = \frac{1}{2} \sum_{t,k} (y_k^t - y_k^{*,t})^2$, targets $y_k^{*,t}$ and outputs y_k^t as defined in Equation (11) in Methods. The partial derivative $\frac{\partial E}{\partial z_j^t}$ takes the form:

$$E = \frac{1}{2} \sum_{t,k} (y_k^t - y_k^{*,t})^2 \quad (13)$$

$$\frac{\partial E}{\partial z_j^t} = \sum_k W_{kj}^{\text{out}} \sum_{t' \geq t} (y_k^{t'} - y_k^{*,t'}) \kappa^{t'-t} . \quad (14)$$

This seemingly provides an obstacle for online learning, because the partial derivative is a weighted sum over future errors. But this problem can be resolved. Following Equation (1) in Results, the approximation $\widehat{\frac{dE}{dW_{ji}}}$ of the loss gradient is computed with e-prop as follows (we insert $\frac{\partial E}{\partial z_j^t}$ in place of the total derivative $\frac{dE}{dz_j^t}$):

$$\widehat{\frac{dE}{dW_{ji}}} = \sum_{t'} \frac{\partial E}{\partial z_j^{t'}} e_{ji}^{t'} \quad (15)$$

$$= \sum_{k,t'} W_{kj}^{\text{out}} \sum_{t \geq t'} (y_k^t - y_k^{*,t}) \kappa^{t-t'} e_{ji}^{t'} \quad (16)$$

$$= \sum_{k,t} W_{kj}^{\text{out}} (y_k^t - y_k^{*,t}) \underbrace{\sum_{t' \leq t} \kappa^{t-t'} e_{ji}^{t'}}_{\stackrel{\text{def}}{=} \tilde{e}_{ji}^t} , \quad (17)$$

where we changed the order of summations in the last line. The second sum indexed by t' is now over previous events that can be computed online. It is just a low-pass filtered version of the eligibility trace e_{ji}^t . With this additional filtering of the eligibility trace with a time constant equal to that of the leak of output neurons, we see that e-prop takes into account the latency between an event at time t' and its impact on later errors at time t within the integration time window of the output neuron. Hence, implementing weight updates with gradient descent and learning rate η , the plasticity rule resulting from e-prop is given by the Equation (28) in Methods. The gradient of the loss function with respect to the output weights $\frac{dE}{dW_{kj}^{\text{out}}}$ can be implemented online without relying on the theory of e-prop. The plasticity rule resulting from gradient descent is directly:

$$\Delta W_{kj}^{\text{out}} = -\eta \sum_t (y_k^t - y_k^{*,t}) \mathcal{F}_\kappa(z_j^t) . \quad (18)$$

Similarly the update of the bias of the output neurons is $\Delta b_k^{\text{out}} = -\eta \sum_t (y_k^t - y_k^{*,t})$.

Classification tasks: We assume that K target categories are provided in the form of a K -dimensional one-hot encoded vector $\pi^{*,t}$. To train recurrent networks in this setup, we replace the mean squared error by the cross entropy loss:

$$E = - \sum_{t,k} \pi_k^{*,t} \log \pi_k^t, \quad (19)$$

where the probability for class k predicted by the network is given as $\pi_k^t = \text{softmax}_k(y_1^t, \dots, y_K^t) = \exp(y_k^t) / \sum_{k'} \exp(y_{k'}^t)$. To derive the modified learning rule that results from this loss function E , we replace $\frac{\partial E}{\partial z_j^t}$ of Supplementary Equation (14) above with the corresponding one resulting from (19):

$$\frac{\partial E}{\partial z_j^t} = \sum_k W_{kj}^{\text{out}} \sum_{t' \geq t} (\pi_k^{t'} - \pi_k^{*,t'}) \kappa^{t'-t}. \quad (20)$$

Following otherwise the same derivation as above in Supplementary Equations (15)-(17), the plasticity rule in the case of classification tasks is given in Methods by Equation (29).

Similarly, one obtains the plasticity rule for the output connections, where the only difference between the cases of regression and of classification is that the output y_k^t and the target $y_k^{*,t}$ are replaced by π_k^t and $\pi_k^{*,t}$ respectively: $\Delta W_{kj}^{\text{out}} = -\eta \sum_t (\pi_k^t - \pi_k^{*,t}) \mathcal{F}_\kappa(z_j^t)$. The update of the bias of the output neurons is $\Delta b_k^{\text{out}} = -\eta \sum_t (\pi_k^t - \pi_k^{*,t})$.

Simulation details: framewise phoneme recognition task (Figure 2)

The goal of the frame-wise setup of the task is to classify audio-frames into phoneme classes. Every input sequence of audio-frames has a corresponding sequence of class labels of the same length, hence the model does not need to align the input sequence to the target sequence. This task has been widely adopted as a phoneme recognition benchmark for recurrent neural networks (RNNs).

Details of the network model: In Figure 2c the performance is reported for the same network architecture as used for LSTM networks in [2], it uses in particular a bi-directional architecture where the output of the LSNN is augmented by the output of a second LSNN that receives the input sequence in reverse time order. This reduces the error in comparison to the uni-directional case from 36.1% to 32.9% for LSNNs with BPTT. This improvement is qualitatively similar to what was previously reported for LSTM networks [2].

With the bi-directional architecture used in Figure 2c, each of the two networks consisted of 300 LIF neurons and 100 ALIF neurons. The neurons in the LSNN had a membrane time constant of $\tau_m = 20$ ms, an adaptation time constant of $\tau_a = 200$ ms, an adaptation strength of $\beta = 0.184$, a baseline threshold $v_{\text{th}} = 1.6$, and a refractory period of 2 ms.

We used 61 output neurons in total, one for each class of the TIMIT dataset. The membrane time constant of the output neurons was $\tau_{\text{out}} = 3$ ms. A softmax was applied to their output, resulting in the corresponding class probabilities. The network model had ≈ 0.4 million weights.

Details of the dataset preparation and of the input preprocessing: We followed the same task setup as in [4, 2]. The TIMIT dataset was split according to [10] into a training, validation, and test set with 3696, 400, and 192 sequences respectively. The input $\mathbf{x}^t$ was given as preprocessed audio that was obtained by the following procedure: Computation of 13 Mel Frequency Cepstral Coefficients (MFCCs) with a frame size of 10 ms on an input window of length 25 ms, computation of the first and the second derivatives of MFCCs, concatenation of all computed factors. The 39 input channels were mapped to the range $[0, 1]$ according to the minimum/maximum values in the training set.

In order to map the inputs into the temporal time domain of LSNNs, each preprocessed audio frame was fed as inputs $\mathbf{x}^t$ to the LSNN for 5 consecutive 1 ms steps.

Details of the learning procedure: All networks were trained for a maximum of 80 epochs, where we used early stopping to report the test error at the point of the lowest error on the validation set. Weight updates were implemented using Adam with default hyperparameters [6] except for ϵ_{Adam} , which was set to 10^{-5} . Gradients were computed using batches of size 32. We used L2 regularization in all networks by adding the term $10^{-5} \cdot \|W\|^2$ to the loss function, where W denotes all weights in the network. The learning rate was initialized to 0.01 and fixed during training. For random e-prop and adaptive e-prop, broadcast weights B_{jk} were initialized using a Gaussian distribution with a mean of 0 and a variance of 1 and $1/n$ (n being the number of neurons in the LSNN) respectively. In adaptive e-prop, we used in addition to the weight decay described above L2 weight decay on output and broadcast weights according to 2 using a factor of $c_{\text{decay}} = 10^{-2}$. Firing rate regularization, as described in section 2, was applied with $c_{\text{reg}} = 50$.

Simulation details: phoneme sequence recognition with CTC (Figure 2)

We compared e-prop and BPTT on the task and the network architecture used in [3]. The essential building blocks of this architecture were also used in [11] for developing commercial software for speech-to-text transcriptions. In this architecture Connectionist Temporal Classification (CTC) is employed. This enabled us to train networks on unaligned sequence labeling tasks end-to-end. We considered the results of [3] that were obtained with three layers of bi-directional LSTM networks, CTC, and BPTT as a reference. We are aware that this configuration cannot be adapted to an online implementation easily, due to the usage of a bi-directional LSTM network and the CTC loss function. However, we believe that this task is still relevant to compare BPTT and e-prop because it is a well established benchmark for RNNs.

Details of the network model: The neurons were structured into 3 layers. The network was recurrently connected within a layer and had feedforward connections across layers. Each layer consisted of 80 LIF neurons and 720 ALIF neurons (9.1 million weights). The neurons in LSNNs had a membrane time constant of $\tau_m = 20$ ms, an adaptation time constant of $\tau_a = 500$ ms, an adaptation strength of $\beta = 0.074$, a baseline threshold $v_{\text{th}} = 0.2$, and a refractory period of 2 ms. Synaptic delays were randomly chosen from $\{1, 2\}$ ms with equal probability. The membrane time constant of output neurons was $\tau_{\text{out}} = 3$ ms.

E-prop with many layers of recurrent neurons: If one naively applies e-prop in such a configuration, the partial derivative $\frac{\partial E}{\partial z_j^t}$ is non-zero only if j belongs to the last layer, whereas earlier layers would not receive any learning signal. To avoid this, we connected all neurons in all layers of the RNN to the output neurons. Therefore, the outputs y_k^t of the RNN were given as $y_k^t = \sum_{t' \leq t} \kappa^{t-t'} \sum_l \sum_j W_{kj}^{\text{out},(l)} z_j^{(l),t'}$, where $z_j^{(l),t'}$ denotes the visible state of a neuron j within the layer l . As a result, the learning signals in the case of e-prop were non-zero for neurons in every layer.

E-prop with the CTC loss function: E_{CTC} is defined based on the log-likelihood of obtaining the sequence of labeled phonemes given the network outputs y_k^t . We refer to [12] for the formal definition of the probabilistic model. Equation (7.27) in [13] shows the gradient of the loss function E_{CTC} with respect to the activity of the outputs y_k^t that we denote as $\frac{dE}{dy_k^t}$. Using the linear relationship between the visible state $z_j^{(l),t}$ and the outputs y_k^t , we obtain that the partial derivative $\frac{\partial E_{\text{CTC}}}{\partial z_j^{(l),t}}$ that we need in order to find the learning signals used in e-prop, and are defined as $\sum_{t' \geq t} \kappa^{t'-t} \sum_k \frac{dE}{dy_k^{t'}} B_{jk}^{(l)}$. Here, $B_{jk}^{(l)}$ denote the broadcast weights to the layer l .

Details of the dataset preparation and of the input preprocessing: The TIMIT dataset was split in the same manner as in the frame-wise version of the task. The raw audio was preprocessed before it was provided as an input $\mathbf{x}^t$ to the network. This included the following

steps: computation of a Fourier-transform based filter-bank with 40 coefficients and an additional channel for the signal energy (with step size 10 ms and window size 25 ms), computation of the first and the second derivatives, concatenation of all computed factors, which totals to 123 input channels. Normalization over the training set was done in the same manner as in the frame-wise version of the task.

In order to map the inputs into the temporal time domain of LSNNs, each preprocessed audio frame was fed as inputs $\mathbf{x}^t$ to the LSNN for 5 consecutive 1 ms steps.

Details of the learning procedure: All models were trained for a total of 60 epochs, where gradients were computed using batches of 8 sequences. The learning rate was initialized to 10^{-3} and decayed every 15 epochs by a factor of 0.3. We used early stopping to report the test error, as in the previous task. Dropout was applied during training between the hidden layers and at the output neurons with a dropout probability of 0.3. As in the frame-wise setup, the weight updates were implemented using Adam with the default hyperparameters [6] except for $\epsilon_{\text{Adam}} = 10^{-5}$. For random e-prop and adaptive e-prop, broadcast weights B_{jk} were initialized using a Gaussian distribution with a mean of 0 and a variance of 1 and $1/n$ (n denoting the number of neurons in the corresponding layer) respectively. In adaptive e-prop, we used L2 weight decay on output and broadcast weights according to 2 using a factor of $c_{\text{decay}} = 10^{-4}$. When the global norm of gradients $N_{\text{clip}} = \|\widehat{\frac{dE}{dW_{ji}^{\text{in}}}}\|^2 + \|\widehat{\frac{dE}{dW_{ji}^{\text{rec}}}}\|^2 + \|\widehat{\frac{dE}{dW_{ji}^{\text{out}}}}\|^2$ was larger than 1, we scaled the gradients by a factor of $\frac{1}{N_{\text{clip}}}$. We used beam search decoding with a beam width of 100. As in [3], the networks were trained on all 61 phoneme labels but were then mapped to a reduced phoneme set (39 classes) for testing.

Applying e-prop to an episodic memory task

The FORCE training method [1] arguably defines the state of the art for training methods for RSNNs that do not need to backpropagate gradients through time. FORCE learning uses a synaptic plasticity rule that requires knowledge of the values of all synaptic weights in the network. This rule was not argued to be biologically plausible, but no other methods for training an RSNN to solve the task described below were known so far.

In order to compare e-prop to FORCE learning, we tested e-prop on the task to replay a movie segment that was used as a target $\mathbf{y}^{*,t}$ [1]. Specifically, it had to generate at each time step the values of all pixels that described the video frame of the movie at that time step. This episodic memory task was arguably the most difficult task for which an RSNN was previously trained in [1].

Here, we considered an extension to this task: An LSNN had to replay 1 out of 3 possible movies, where the desired movie index was provided as a cue to the network, see Supplementary Figure 1a. As in [1], the LSNN received also a clock-like input signal to indicate the current position in the movie. We show in Supplementary Figure 1b that an LSNN can be trained to solve this task with either one of the e-prop versions and that e-prop performs almost as well as BPTT.

Details of the network model: We used an LSNN that consisted of 700 LIF neurons and 300 ALIF neurons. Each neuron had a membrane time constant of $\tau_m = 20$ ms and a refractory period of 5 ms. ALIF neurons had a threshold adaptation time constant of 500 ms, and a threshold adaptation strength of $\beta = 0.07$. All neurons had a baseline threshold of $v_{\text{th}} = 0.62$. All 5544 output neurons had a membrane time constant of $\tau_{\text{out}} = 4$ ms.

Details of the dataset preparation and of the input scheme: We manually chose three movie clips from the Hollywood 2 dataset [14], which contained between 0 and 2 scene cuts*. The movie clips were clipped to a length of 5 seconds and spatially subsampled to a resolution of

*sceneclipautoautotrain00019.avi, sceneclipautoautotrain00061.avi, sceneclipautoautotrain00071.avi

66×28 pixels. Since our simulations used 1 ms as a discrete time step, we linearly interpolated between the frames of the original movie clips, which had a framerate of 25 frames per second. In total, we obtained a target signal with $66 \times 26 \times 3 = 5544$ dimensions, whose values were divided by a constant of 255, such that they fit in the range of $[0, 1]$.

The network received input from 115 input neurons, divided into 23 groups of 5 neurons. The first 20 groups indicated the current phase of the target sequence, similar to [1]. Neurons in group $i \in \{0, 19\}$ produced regular spike trains with a firing rate of 50 Hz during the time interval $[250 \cdot i, 250 \cdot i + 250)$ ms and were silent at other times. The remaining 3 groups encoded which movie had to be replayed, where each group was assigned to one of the three movies. To indicate a desired replay of one specific movie, each neuron in the corresponding group produced a Poisson spike train with a rate of 50 Hz and was silent otherwise.

Details of the learning procedure: For learning, we carried out 5 second simulations, where the network produced a 5544 dimensional output pattern. Gradients were accumulated for 8 successive trials, after which weight updates were applied using Adam with a learning rate of $2 \cdot 10^{-3}$ and default hyperparameters [6]. The movie to be replayed in each trial was selected with uniform probability. After every 100 weight updates (iterations), the learning rate was decayed by a factor of 0.95. For random e-prop, we used random broadcast weights B_{jk} that were sampled from a Gaussian distribution with a mean of 0 and a variance of 1. In adaptive e-prop we used L2 weight decay (see Supplementary Note 2 on weight decay) for the broadcast weights B_{jk} and the output weights W_{ji}^{out} with a factor of $c_{\text{decay}} = 0.001$. To avoid an excessively high firing rate, regularization, as described in Supplementary Note 2 on firing rate regularization, was applied with $c_{\text{reg}} = 0.1$ and a target firing rate of $f^{\text{target}} = 10$ Hz.

Simulation details: task where temporal credit assignment is difficult (Figure 3)

This task was inspired by the task performed by rodents in [15]. Each trial was split into three periods: the cue period, the delay period, and the decision period. During the cue period, the subject was stimulated with 7 successive binary cues (“left” or “right”), and had to take a corresponding binary decision (“left” or “right”) during the decision period. The trial was considered a success if the decision matched the side that was most often indicated by the 7 cues. No action was required during the delay period. Each cue lasted for 100 ms and the cues were separated by 50 ms. The duration of the delay was distributed uniformly between 500 ms and 1500 ms, and the decision period lasted for 150 ms. The results from Figure 3 were averaged over 20 different runs, where each run was characterized by a different random seed. For each run this resulted in different initialization of the weight matrices and broadcast weights as well as different realizations of random inputs.

Details of the network model and input scheme: We used an LSNN that consisted of 50 LIF neurons and 50 ALIF neurons. All neurons had a membrane time constant of $\tau_m = 20$ ms, a baseline threshold of $v_{\text{th}} = 0.6$, and a refractory period of 5 ms. The time constant of the threshold adaptation was set to $\tau_a = 2000$ ms, and its impact on the threshold was given as $\beta = 1.74 \cdot 10^{-2}$.

Input to this network was provided by 4 populations of 10 neurons each. The first two input populations encoded the cues as follows: When a cue indicated the “left” side (resp. the “right” side), all the neurons within the first (resp. the second) population produced Poisson spike trains with a firing rate of 40 Hz. The third input population spiked randomly throughout the decision period with a firing rate of 40 Hz and was silent otherwise. All the neurons in the last input population produced Poisson spike trains of 10 Hz throughout the trial, which was useful in particular to avoid that the network becomes quiescent during the delay.

Details of the learning procedure: For learning, we used e-prop for classification tasks, see Supplementary Note 3. The target label $\pi_k^{*,t}$ was given as the correct output during the decision

period at the end of a trial. To help the network solve the task, we used a curriculum with an increasing number of cues. We first trained with a single cue, and increased the number of cues to 3, 5 and finally 7. The number of cues increased each time the network achieved less than 8% error on 512 validation trials. The same criterion was used to stop training once 7 cues were reached.

Independent of the learning algorithm that was used (BPTT, e-prop), a weight update was applied once every 64 trials and the gradients were accumulated during those trials additively. All weight updates were implemented using Adam with default parameters [6] and a learning rate of $5 \cdot 10^{-3}$. In the cases of random e-prop and adaptive e-prop, broadcast weights B_{jk} were initialized using a Gaussian distribution with mean 0 and variance 1. In adaptive e-prop we used L2 weight decay (see Supplementary Note 2) for the broadcast weights B_{jk} and the output weights W_{ji}^{out} with a factor of $c_{\text{decay}} = 0.001$. In addition, firing rate regularization, as described in the Supplementary Note 2, was applied with $c_{\text{reg}} = 1$. and a target firing rate of $f^{\text{target}} = 10$ Hz.

Details of the control experiment in Supplementary Figure 8: For this control experiment, we introduced 1 to 3 additional feedforward (recurrent connections disabled) layers of 50 LIF neurons each before the output neurons. Each layer was connected in an all-to-all fashion to the preceding layer. All neurons had a membrane time constant of $\tau_m = 20$ ms. There was no voltage or firing rate regularization applied to the feedforward layers. We also applied the same curriculum of an increasing number of cues as described in the preceding section. In addition each stage of the curriculum was restricted to 1000 weight updates. All other hyperparameters were identical to the preceding section.

Analysis of information in adaptive firing thresholds: We verified (using 64 runs with 128 trials each for testing) that the firing threshold during the delay (500 ms after the last input cue) contained a substantial part of the decision-relevant information. The correct decision could be decoded by a simple readout network (1 hidden layer consisting of 100 ReLU neurons, trained over 200 iterations with 128 trials each) from the current values of the firing thresholds of ALIF neurons with an accuracy of $95.6\% \pm 1.82\%$ (standard deviation). From low pass filtered spikes of LIF neurons at the end of the recall period it could be decoded with an accuracy of $86.6\% \pm 3.7\%$ (standard deviation).

4 Applying supervised learning with e-prop to artificial neural networks (LSTM networks)

Here we show that e-prop can also be applied to artificial neural networks. We chose long short-term memory (LSTM) networks [16] for this demonstration, whose performance defines the standard for RNNs in machine learning. We demonstrate below that LSTM networks can achieve competitive results on TIMIT when trained with e-prop, followed by details on these simulations. In the following sections, we provide details on the LSTM network model used and on eligibility traces for LSTM units.

Phoneme recognition with LSTM networks and e-prop

In Results, we have used e-prop to train LSNNs on the phoneme recognition task TIMIT (see Figure 2). To test whether e-prop is effective also for artificial neural networks, we applied it to LSTM networks on the very same task in its two flavors of frame-wise classification and sequence transcription.

Supplementary Figure 4 shows that e-prop approximates the performance of BPTT in both versions of TIMIT also for LSTM networks very well. For the more difficult version of TIMIT involving sequence transcription, we trained a feedforward sequence of 3 LSTM networks as in [3].

Simulation details: framewise phoneme recognition task with LSTM networks (Supplementary Figure 4)

The data preparation in the two setups (frame-wise phoneme classification and phoneme sequence recognition) were identical to the LSNN case. They are described in Supplementary Note 3. The details on the network models and training procedures are described next for the two task setups separately.

Details of the network model: We used a bi-directional network architecture [2], where the output of an LSTM network was augmented by the output a second LSTM network that received the input sequence in reverse time order. Each of the two networks consisted of 200 LSTM units.

We used a 61-fold softmax output, one for each class of the TIMIT dataset. The LSTM network had ≈ 0.4 million weights, which matched the number of weights in the LSNN for the same task.

Details of the learning procedure: LSTM networks were trained in the same way as LSNNs, see Supplementary Note 3, except for the following differences in hyperparameters: We decayed the learning rate after every 500 weight updates by a factor of 0.3. For L2 weight decay on output and broadcast weights according to 2 we used a factor of $c_{\text{decay}} = 10^{-3}$. As LSTM units are not spiking, we did not use firing rate regularization.

Simulation details: phoneme sequence recognition with CTC and LSTM networks (Supplementary Figure 4)

We compared e-prop and BPTT on the task and the network architecture used in [3]. As for LSNNs, we employed Connectionist Temporal Classification (CTC) to achieve phoneme sequence recognition (see “Phoneme sequence recognition with CTC” in Supplementary Note 3). This enabled us to train networks on unaligned sequence labeling tasks end to end.

Details of the network model: The neurons were structured into 3 recurrent layers. In each layer there were 250 LSTM units. All neurons in all layers of the RNN were connected to the output layer (see “E-prop with many layers of recurrent neurons” in Supplementary Note 3).

Details of the learning procedure: LSTM networks were trained in the same way as LSNNs, see Supplementary Note 3. In the case of BPTT, we also used the peephole feature in the LSTM network model.

LSTM network model

We used a standard model for LSTM units [16], for which the hidden state at time step t is a one dimensional vector containing only the content of the memory cell c_j^t , such that $\mathbf{h}_j^t \stackrel{\text{def}}{=} [c_j^t]$, and z_j^t is the value of its output. The memory cell can be viewed as a register which supports writing, updating, deleting and reading. These operations are controlled independently for each cell j at each time t by input, forget and output gates (denoted by i_j^t , f_j^t and o_j^t respectively). The new cell state candidate that may replace the cell state c_j^{t-1} at each time step t is denoted $\tilde{c}_j^t$. The input, forget, and output sigmoidal gates as well as the cell state candidate of an LSTM unit j

are defined by the following equations:

$$\dot{c}_j^t = \sigma\left(\sum_i W_{ji}^{\text{rec},\dot{c}} z_i^{t-1} + \sum_i W_{ji}^{\text{in},\dot{c}} x_i^t\right) \quad (21)$$

$$\dot{\ell}_j^t = \sigma\left(\sum_i W_{ji}^{\text{rec},\dot{\ell}} z_i^{t-1} + \sum_i W_{ji}^{\text{in},\dot{\ell}} x_i^t\right) \quad (22)$$

$$\dot{o}_j^t = \sigma\left(\sum_i W_{ji}^{\text{rec},\dot{o}} z_i^{t-1} + \sum_i W_{ji}^{\text{in},\dot{o}} x_i^t\right) \quad (23)$$

$$\tilde{c}_j^t = \tanh\left(\sum_i W_{ji}^{\text{rec},c} z_i^{t-1} + \sum_i W_{ji}^{\text{in},c} x_i^t\right), \quad (24)$$

where all the weights used here are parameters of the model (we also used biases that were omitted for readability). Using these notations, one can now write the update of the state of an LSTM unit j in a form that we can relate to our general formalism:

$$c_j^t = \dot{\ell}_j^t c_j^{t-1} + \dot{c}_j^t \tilde{c}_j^t \quad (25)$$

$$z_j^t = \dot{o}_j^t \tanh(c_j^t). \quad (26)$$

In terms of the computational graph in Figure 6, Supplementary Equations (25) and (26) define $M(c_j^{t-1}, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$ and $f(c_j^t, \mathbf{z}^{t-1}, \mathbf{x}^t, \mathbf{W}_j)$.

Eligibility traces for LSTM units

Eligibility traces for LIF neurons and ALIF neurons were derived in “Derivation of eligibility traces for concrete neuron models” in Methods. Here, we derive eligibility traces for the weights of LSTM units.

To obtain the eligibility traces, we note that the state dynamics of an LSTM unit is given by: $\frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} = \frac{\partial c_j^t}{\partial c_j^{t-1}} = \dot{\ell}_j^t$. For each weight $W_{ji}^{A,B}$ with A being either “in” or “rec” and B being $\dot{c}$, $\dot{\ell}$, or c , we compute a set of eligibility traces. For example, the eligibility vectors for the recurrent weights to the input gate $W_{ji}^{\text{rec},\dot{c}}$, are updated according to Equation (14) in Methods, leading to:

$$\epsilon_{ji}^{\dot{c},t} = \dot{\ell}_j^t \epsilon_{ji}^{\dot{c},t-1} + \tilde{c}_j^t \dot{c}_j^t (1 - \dot{c}_j^t) z_i^{t-1}, \quad (27)$$

resulting in eligibility traces:

$$e_{ji}^{\dot{c},t} = \dot{o}_j^t \tanh'(c_j^t) \epsilon_{ji}^{\dot{c},t}, \quad (28)$$

where $\tanh'(c_j^t) = 1 - \tanh^2(c_j^t)$ is the derivative of $\tanh$. Similarly, the eligibility traces for the input weights to the input gate are obtained by replacing z_i^{t-1} with x_i^t .

Output gates: The gradients with respect to the parameters of the output gate do not require additional eligibility traces. This is because the output gate contributes to the observable state but not to the hidden state, see Supplementary Equations (25) and (26). Therefore, one can use the standard factorization of the error gradient as used in BPTT. For the recurrent weights to the output gates $W_{ji}^{\text{rec},\dot{o}}$, the gradient is given by:

$$\frac{dE}{dW_{ji}^{\text{rec},\dot{o}}} = \sum_t \frac{dE}{dz_j^t} \frac{\partial z_j^t}{\partial W_{ji}^{\text{rec},\dot{o}}} = \sum_t \frac{dE}{dz_j^t} \tanh'(c_j^t) \dot{o}_j^t (1 - \dot{o}_j^t) z_i^{t-1}. \quad (29)$$

Hence, when applying e-prop to the weights of the output gate of an LSTM unit, we use, similarly as in Equation (27) from Methods, the same approximation $\frac{\partial E}{\partial z_j^t}$ of the ideal learning signal $\frac{dE}{dz_j^t}$. The other term is local, depending only on t and $t-1$, and does not require eligibility traces. For input weights to the output gate $W_{ji}^{\text{in},\dot{o}}$, the gradient is obtained by replacing z_i^{t-1} with x_i^t .

5 Reward-based e-prop: Application of e-prop to deep RL

Synaptic plasticity rules for reward-based e-prop

Here, we derive the synaptic plasticity rules that result from gradients of the loss function E , as given in Methods in Equation (32), see Figure 4b for the network architecture. As a result of the general actor-critic framework with policy gradient, this loss function additively combines the loss function for the policy E_π (actor) and the value function E_V (critic).

We consider two cases: First, a simplified case where in each trial, one action is taken at the end of the trial. This is the setup of the reward-based version of the task where temporal credit assignment is difficult of Figure 3, see Supplementary Figure 5 for performance results. Second, we analyse a more general case where actions are taken throughout the trial. This is the setup of the Atari tasks (Figure 4, Figure 5). For both cases, we derive the gradients for the parts E_π and E_V of the loss function E , and express the plasticity rules resulting from these gradients.

Single action at the end of the trial (Supplementary Figure 5): In this setup, a discrete action $a \in \{1, \dots, K\}$ from a set of K possibilities needs to be taken at the last time step T of a trial, leading to a binary-valued reward r^T . As a result, the return R^T (denoted here for notational simplicity as R) is equal to r^T . We assume that the agent chooses action k with probability $\pi_k = \text{softmax}_k(y_1^T, \dots, y_K^T) = \exp(y_k^T) / \sum_{k'} \exp(y_{k'}^T)$. Following the definition of E_π in (30) in Methods one can write the loss function E_π using the one-hot encoded action $\mathbb{1}_{a=k}$ which assumes a value of 1 only if $a = k$ and is 0 otherwise. Hence we can write $\log \pi(a|\mathbf{y}) = \sum_k \mathbb{1}_{a=k} \log \pi_k$, where although we sum over all possible actions, only the term corresponding to the action a that was taken is non zero. Therefore, we can write E_π as:

$$E_\pi = -R \log \pi(a|\mathbf{y}) = -R \sum_k \mathbb{1}_{a=k} \log \pi_k. \quad (30)$$

Interestingly, in the discrete action case, the loss function E_π is reminiscent of the one used for supervised classification, see Supplementary Equation (19) above. But it exhibits two differences: firstly, the indicator of the selected action $\mathbb{1}_{a=k}$ replaces the target label π_k^* , and secondly, the loss is multiplied by the reward R . Hence, the derivation of the weight update is almost identical with the additional difference that we also replace R with $R - V$ in the resulting gradient to reduce the variance of the loss gradient estimate as derived in the general case in (35) from Methods. Finally, in order to optimize E defined in (32) in Methods, we also need to consider $E_V = \frac{1}{2}(R - V)^2$, for which we can reuse the weight update of e-prop for regression given above in (17). Combining the weight updates for E_π and E_V into a single one, we have:

$$\Delta W_{ji}^{\text{rec}} = -\eta \underbrace{\left[(R - V) \sum_k B_{jk}^\pi (\pi_k - \mathbb{1}_{a=k}) - c_V (R - V) B_j^V \right]}_{L_j} \tilde{e}_{ji}^T, \quad (31)$$

where we denote with B_{jk}^π the broadcast weights from output neurons y_k , and with B_j^V the broadcast weights from the output neuron that produces the value prediction V . The choice of these broadcast weights then defines which variant of reward-based e-prop is employed (reward-based symmetric e-prop, reward-based adaptive e-prop, or reward-based random e-prop).

For the synaptic connections of output neurons, the loss gradient can be computed directly from the loss function (32) in Methods. We also subtract the value prediction to reduce variance of the gradient estimate as in (35) in Methods, and obtain for the update rules: $\Delta W_{kj}^{\pi, \text{out}} = -\eta(R - V)(\pi_k - \mathbb{1}_{a=k})\mathcal{F}_\kappa(z_j^T)$, and $\Delta W_j^V = \eta c_V (R - V)\mathcal{F}_\kappa(z_j^T)$. Similarly, the updates of the biases of output neurons are: $\Delta b_k^{\pi, \text{out}} = -\eta(R - V)(\pi_k - \mathbb{1}_{a=k})$, and $\Delta b^V = \eta c_V (R - V)$.

Actions throughout the trial (Figure 4 and 5): In this setup, we assume that the agent can take discrete actions a_k among K possible choices at certain decision times $t_0, \dots, t_n, \dots$

We also assume that the agent chooses action k with a probability $\pi_k^{t_n}$ based on a categorical distribution that depends on the network output: $\pi_k^{t_n} = \pi(a_k | \mathbf{y}^{t_n}) = \text{softmax}_k(y_1^{t_n}, \dots, y_K^{t_n}) = \exp(y_k^{t_n}) / \sum_{k'} \exp(y_{k'}^{t_n})$.

To derive the gradient estimate resulting from reward-based e-prop, we first consider the regression problem defined by the loss function E_V , and note that a major difference to the previous case is that the return R^t integrates future rewards that may arrive long after an action was taken. We begin with the result for regression given above in Supplementary Equation (17). Substituting the relevant variables, we obtain an estimation of the loss gradient:

$$\widehat{\frac{dE_V}{dW_{ji}^{\text{rec}}}} = - \sum_{t'} (R^{t'} - V^{t'}) W_j^{V, \text{out}} \bar{e}_{ji}^{t'}, \quad (32)$$

where $W_j^{V, \text{out}}$ are the weights of the output neuron V_j^t predicting the value function $\mathbb{E}[R^t]$. In order to overcome the obstacle that an evaluation of the return $R^{t'}$ requires to know future rewards, we introduce temporal difference errors $\delta^t = r^t + \gamma V^{t+1} - V^t$, and use that $R^{t'} - V^{t'}$ is equal to the sum $\sum_{t \geq t'} \gamma^{t-t'} \delta^t$. We then reorganize the two sums over t and t' (note that the interchange of the summation order amounts to the equivalence between forward and backward view of RL [17]):

$$\widehat{\frac{dE_V}{dW_{ji}^{\text{rec}}}} = - \sum_{t'} \left(\sum_{t \geq t'} \gamma^{t-t'} \delta^t \right) W_j^{V, \text{out}} \bar{e}_{ji}^{t'} \quad (33)$$

$$= - \sum_t \delta^t \sum_{t' \leq t} \gamma^{t-t'} W_j^{V, \text{out}} \bar{e}_{ji}^{t'} \quad (34)$$

$$= - \sum_t \delta^t \mathcal{F}_\gamma(W_j^{V, \text{out}} \bar{e}_{ji}^t). \quad (35)$$

For the part E_π in the loss function E defined at Equation (30) in Methods, we consider the estimator $\widehat{\frac{\partial E}{\partial z_j^t}}$ given in (35) in Methods, and use our previous definition that the probabilities of actions a_k follow a categorical distribution, using a softmax of the network output y_k . The estimator then becomes:

$$\widehat{\frac{\partial E_\pi}{\partial z_j^t}} = \sum_k W_{kj}^{\pi, \text{out}} \sum_{\{n \mid t_n \geq t\}} \kappa^{t_n - t} (R^{t_n} - V^{t_n}) (\pi_k^{t_n} - \mathbb{1}_{a^{t_n}=k}), \quad (36)$$

where $W_{kj}^{\pi, \text{out}}$ are the weights onto the output neurons y_k^t defining the policy π , and κ is the constant of the low-pass filtering of the output neurons. Following a derivation similar to Supplementary Equations (15) to (17) given above for supervised regression, we arrive at an estimation of the loss gradient of the form:

$$\widehat{\frac{dE_\pi}{dW_{ji}^{\text{rec}}}} = \sum_t \widehat{\frac{\partial E_\pi}{\partial z_j^t}} e_{ji}^t \quad (37)$$

$$= \sum_{t, k} W_{kj}^{\pi, \text{out}} \sum_{\{n \mid t_n \geq t\}} (R^{t_n} - V^{t_n}) (\pi_k^{t_n} - \mathbb{1}_{a^{t_n}=k}) \kappa^{t_n - t} e_{ji}^t \quad (38)$$

$$= \sum_{n, k} (R^{t_n} - V^{t_n}) W_{kj}^{\pi, \text{out}} (\pi_k^{t_n} - \mathbb{1}_{a^{t_n}=k}) \underbrace{\sum_{t \leq t_n} \kappa^{t_n - t} e_{ji}^t}_{\bar{e}_{ji}^{t_n}}. \quad (39)$$

Like in the derivation of the gradient of E_V , this formula hides a sum over future rewards in R^{t_n} that cannot be computed online. It is resolved by introducing the backward view as in Equation (35) above. We arrive at the loss gradient:

$$\widehat{\frac{dE_\pi}{dW_{ji}^{\text{rec}}}} = \sum_t \delta^t \mathcal{F}_\gamma \left(\sum_k W_{kj}^{\pi, \text{out}} (\pi_k^t - \mathbb{1}_{a^t=k}) \bar{e}_{ji}^t \right). \quad (40)$$

Importantly, an action is only taken at times $t_0, \dots, t_n, \dots$, hence for all other times, we set the term $\pi_k^t - \mathbb{1}_{a^t=k}$ to zero.

Finally, the gradient of the loss function E is the sum of the gradients of E_π and E_V in Equations (35) and (40) from Supplementary Materials. Application of stochastic gradient descent with a learning rate of η yields the synaptic plasticity rule given in the Equations (36) and (37) in Methods.

Policy entropy regularization: In order to prevent premature convergence and encourage exploration, we add to the loss function of reward-based e-prop an auxiliary loss that maximizes the entropy H of the policy by $E_H = -\sum_t H(\pi_1^t, \dots, \pi_K^t) \stackrel{\text{def}}{=} -\sum_t H^t$. Hence the Equation (35) defining the estimators of the loss gradients $\widehat{\frac{\partial E}{\partial z_j^t}}$ in Methods becomes:

$$\widehat{\frac{\partial E}{\partial z_j^t}} = -\sum_n (R^{t_n} - V^{t_n}) \frac{\partial \log \pi(a^{t_n} | \mathbf{y}^{t_n})}{\partial z_j^t} + c_V \frac{\partial E_V}{\partial z_j^t} + c_H \frac{\partial E_H}{\partial z_j^t}. \quad (41)$$

To exhibit the term $\frac{\partial E_H}{\partial z_j^t}$, we begin with the definition of entropy and find $E_H = \sum_n \sum_k \pi_k^{t_n} \log \pi_k^{t_n} = -\sum_n H^{t_n}$ for the case of actions that probabilities distributed according to categorical distribution:

$$\frac{\partial E_H}{\partial z_j^t} = -\sum_{\{n | t_n \geq t\}} \sum_k \kappa^{t_n-t} W_{kj}^{\pi, \text{out}} \frac{\partial H^{t_n}}{\partial y_k^{t_n}} \quad (42)$$

$$= \sum_{\{n | t_n \geq t\}} \sum_k \kappa^{t_n-t} W_{kj}^{\pi, \text{out}} \pi_k^{t_n} (\log \pi_k^{t_n} + H^{t_n}). \quad (43)$$

And thus, using a similar derivation as in Supplementary Equations (15) to (17), we find the estimate of the gradient of the loss function E_H as:

$$\widehat{\frac{dE_H}{dW_{ji}}} = \sum_t \frac{\partial E_H}{\partial z_j^t} e_{ji}^t = \sum_{n,k} W_{kj}^{\pi, \text{out}} \pi_k^{t_n} (\log \pi_k^{t_n} + H^{t_n}) \bar{e}_{ji}^{t_n}. \quad (44)$$

Hence this adds a term of the form $L_j^{H,t} \bar{e}_{ji}^t$ to the final learning rule of reward-based e-prop exhibited in Equations (36) and (37) in Methods such that it becomes:

$$\Delta W_{ji}^{\text{rec}} = -\eta \sum_t \delta^t \mathcal{F}_\gamma \left(L_j^t \bar{e}_{ji}^t \right) - \eta L_j^{H,t} \bar{e}_{ji}^t \quad \text{with} \quad (45)$$

$$L_j^{H,t} = c_H \sum_k B_{jk}^\pi \pi_k^t (\log \pi_k^t + H^t). \quad (46)$$

Plasticity rule for the output weights and biases: The gradient of E with respect to the output weights can be computed directly from Equation (32) in Methods without the theory of e-prop. However, it also needs to account for the sum over future rewards that is present in the term $R^t - V^t$. Using a similar derivation as above in Supplementary Equations (33)-(35) the plasticity rule for these weights becomes:

$$\Delta W_{kj}^{\pi, \text{out}} = -\eta \sum_t \left[\delta^t \mathcal{F}_\gamma \left((\pi_k^t - \mathbb{1}_{a^t=k}) \mathcal{F}_\kappa(z_j^t) \right) + \pi_k^t (\log \pi_k^t + H^t) \mathcal{F}_\kappa(z_j^t) \right] \quad (47)$$

$$\Delta W_j^{\text{V, out}} = \eta c_V \sum_t \delta^t \mathcal{F}_\gamma \left(\mathcal{F}_\kappa(z_j^t) \right). \quad (48)$$

Similarly, we find the update rules for the biases of the output neurons. We obtain for these: $\Delta b_k^{\pi, \text{out}} = -\eta \sum_t \left[\delta^t \mathcal{F}_\gamma \left((\pi_k^t - \mathbb{1}_{a^t=k}) \right) + \pi_k^t (\log \pi_k^t + H^t) \right]$, and $\Delta b^{\text{V, out}} = \eta c_V \sum_t \delta^t$.

Simulation details: RL version of the task where temporal credit assignment is difficult (Supplementary Figure 5)

The task considered in this experiment was the same as in Figure 3, but while the task was there formulated as a supervised learning, the network is trained here using a reinforcement learning setup. A reward of 1 was given at the end of the trial, if the agent selected the side which had more cues than the other side, otherwise no reward was given. The network model remained the same as in the supervised setup. The result was computed over 10 different runs and is shown in Supplementary Figure 5: The task can be learnt by reward-based e-prop.

Details of the decision process: In the reinforcement learning setup of the task, one binary action formalizes the decision of the agent (“left” or “right”) at the end of the trial. This decision was sampled according to probabilities π_k that were computed from the network output using a softmax operation as described above.

Details of the learning procedure: For learning, we simulated batches of 64 trials, and applied weight changes at the end of each batch. Independent of the learning method, we used Adam to implement the weight update, using gradients that were accumulated in 64 trials using a learning rate of $5 \cdot 10^{-3}$ and default hyperparameters [6]. For random e-prop, we sampled broadcast weights B_{jk} from a Gaussian distribution with a mean of 0 and a variance of 1. To avoid an excessively high firing rate, regularization, as described in Supplementary Note 2, was applied with $c_{\text{reg}} = 0.1$ and a target firing rate of $f^{\text{target}} = 10$ Hz.

Simulation details: Atari task (Figures 4 and 5)

Details of the Atari simulator and of the input scheme: For all our experiments we used the Arcade Learning Environment [18] to simulate various Atari games. This simulator takes the actions from the agent as input and advances the dynamics of the game by repeating the same action for 4 times. The simulator then produces a reward feedback along with the new video frame. The shape of the produced video frame is 160 pixels in width and 210 pixels in height and each pixel has 3 values that describes its color. We first converted the colored video frame into a gray-scale video frame and scaled it to a size of 84 by 84. The gray-scale values (assuming values between 0 and 255) were divided by 255. In our experiments, these observations were each given for 5 ms as input $\mathbf{x}^t, \dots, \mathbf{x}^{t+4}$ to the spiking agent.

Details of evaluation: The results reported in Figure 4 and 5 were computed over 5 different runs, where each run was characterized by a different random seed. For each run this resulted in different random initial weights of the agent and different random numbers for sampling the stochastic action. We chose to compare our spiking agents to the results reported in Table S3 of [19] because of the similarity of the learning approach. Note however that there exist some differences in the evaluation protocol:

1. the results reported in [19] are given as the 5 best runs out of 50, whereas we report the average over 5 runs;
2. their evaluation protocol involves human starting conditions, meaning that evaluation episodes are initialized with human game play, whereas we start episodes randomly with up to 30 No-op actions.

Details of the network model: The network architecture of the agent consisted of a spiking CNN followed by an LSNN. The spiking CNN consisted of 5512 LIF neurons for Pong (10928 LIF neurons for Fishing Derby respectively). These neurons were organized in two layers. Neurons in the first layer received input current resulting from a convolution of the video frame. The

spikes of these neurons were propagated to a second layer of neurons using synapses with a convolutional weight structure. Biases were channel-specific for both convolutions, see Table 1 for all corresponding hyperparameters. All neurons in the two convolutional layers had a membrane time constant of $\tau_{m,\text{CNN}}$ ms and a threshold of $v_{\text{th,CNN}}$, see Table 1. We were forced to use a very high rate of input frames in order to keep the compute time of the learning experiments within the range of our computing resources. Therefore a new image was presented to the network every 5 ms. This forced us to set the membrane constants of the spiking neurons in the preprocessing CNN to the unbiologically low value of 1 ms. We conjecture that a simulation of the same learning processes with larger computing resources, where the presentation time of input frames can be multiplied with some factor F , say $F = 20$, the spiking CNN would be able to process these visual inputs with neurons whose membrane time constants are multiplied with a corresponding factor F , thereby bringing them into a biological range.

The LSNN consisted of 240 LIF neurons and 160 ALIF neurons for Pong (180 LIF neurons and 120 ALIF neurons for Fishing Derby respectively) and received spiking input from all neurons of the second convolutional layer. Membrane time constants τ_m , baseline thresholds v_{th} , adaptation time constants τ_a and adaptation strengths β are reported in Table 1. The refractory period was set to 5 ms. All synaptic delays in the LSNN were 1 ms. The output $\mathbf{y}^t, V^t$ was defined as the membrane voltage of output neurons, whose membrane time constants τ_{out} are reported in Table 1. These output neurons received spikes from all neurons in the network, including neurons in the convolutional layers.

Details of the action set and of the action generation: The set of valid actions was selected according to the game at hand. For Pong, we used a set of 3 valid actions: Up, Down and Stay (No op). For Fishing Derby, we used 9 possible action choices: Stay (No op), Fire, Up, Right, Left, Down, Up-right, Up-Left, Down-right and Down-left. Actions were sampled according to a discrete probability distribution, where the k -th action was sampled with a probability of $\pi_k^t = \text{softmax}_k(y_1^t, \dots, y_K^t)$. Since the agent received the same video frame for 5 ms, actions were also generated only after every 5 ms.

Details of the learning procedure: The agent was trained consuming a total of $250 \cdot 10^6$ action generations, corresponding to 10^9 video frames of the Atari simulator (actions are repeated 4 times internally). We computed gradients resulting from reward-based symmetric e-prop according to Supplementary Equation (45). A common learning scheme is to use actors living in parallel copies of the environment that can regularly communicate gradients with each other during ongoing episodes [19]. But this scheme can hardly provide a model for learning in the brain. For this reason, we designed a schedule of episodes of increasing length and performed weight updates only after 32 completed episodes (64 for Fishing Derby, respectively). A single agent can implement this algorithm through local synaptic plasticity. However this scheme is equivalent to simulating these 32 independent episodes in parallel -but without communication between learning agents- instead of sequentially. This observation can be used to save simulation time.

For Pong the weights of the CNN are only trained to minimize the regularization of the neural activity, hence it does not require learning signals. For Fishing Derby, as every neuron is connected to the output neurons, we define broadcast weights $B_{jk}^{\pi,\text{CNN}}, B_{jk}^{V,\text{CNN}}$ for neurons in the CNN to be symmetric to the output weights: $B_{jk}^{\pi,\text{CNN}} = W_{kj}^{\pi,\text{out,CNN}}, B_j^{V,\text{CNN}} = W_j^{V,\text{out,CNN}}$.

Due to simulation constraints, we reset the fading memory filter $F_\gamma(L_j^t \bar{e}_{ji}^t)$ as well as the eligibility vectors ϵ_{ji}^t to zero after every 500 ms for Pong (150 ms for Fishing Derby respectively). The weight updates were applied using Adam [6] with a learning rate of 10^{-3} (The parameter ϵ was set to 1). In Fishing Derby we clipped these gradients if exceeding a norm of 1000. The maximum episode length T_{max} was restricted to 1000 ms at the start of training, and was increased after every 8750 s by ΔT , see Table 1. We used a discount factor of $\gamma = 0.998$. Note that an action is generated only after every 5 ms. The coefficient c_V that trades off between E_π and E_V can be found in Table 1. We also applied entropy regularization as in Supplementary Equation (44) to prevent premature convergence to suboptimal policies. The value of the coefficient c_H that

adds this loss E_H to the objective is provided in Table 1. To avoid an excessively high firing rate, regularization was applied as described in 2. The target firing rate was given by $f^{\text{target}} = 20$ Hz. We used further regularization to prevent membrane voltages from assuming values for which ψ_j is zero.

6 Evaluation of four variations of *e-prop* (Supplementary Figure 7)

We evaluate here the performance of four variations of random e-prop. In these variations, we used

- truncated eligibility traces for LIF neurons,
- global broadcast weights,
- temporally local broadcast weights, and
- a replacement of the eligibility trace by the corresponding term of the Clopath rule,

respectively. The considered task, whose implementation details are described below 6, is an extension of the task used in [1]. In this task, an RSNN was trained to autonomously generate a 3 dimensional target signal for 1 second. Each dimension of the target signal was given by the sum of four sinusoids with random phases and amplitudes. Similar to [1], the network received a clock input that indicated the current phase of the pattern.

In Supplementary Figure 7a, we show the spiking activity of a randomly chosen subset of 20 out of the 600 neurons in the RSNN along with the output of the three output neurons after application of random e-prop for 1, 100, and 500 seconds, respectively. In this representative example, the network achieved a very good fit to the target signal (normalized mean squared error 0.01).

A truncated eligibility trace for LIF neurons

A replacement of the term $\bar{z}_i^t$ with z_i^t in Equation (23) from Methods yields a performance that is reported in panel b of Supplementary Figure 7 as “Trunc. e-trace”. Its performance is for the considered task only slightly worse than that of random e-prop.

Global broadcast weights

Since 3-factor rules have primarily been studied so far with a global 3rd factor, we asked how the performance of e-prop would change if the same broadcast weight would be used for broadcast connections between all output neurons k and network neurons j . We set this global broadcast weight equal to $\frac{1}{\sqrt{n}}$ with n being the number of neurons in the network. Figure 2 and Supplementary Figure 2 show together that the performance for the considered task is much worse than that of random e-prop. We have also tested this on TIMIT with LSNNs and found there an increase of the frame-wise error rate from 36.9% to 52% when replacing the broadcast weights of random e-prop with a global one. On the harder version of same task, the error rate at the sequence level increased from 34.7% to 60%.

Temporally local broadcast weights

One can train RNNs also by applying the broadcast alignment method of [20] and [21] for feed-forward networks to the unrolled version (see Figure 1b) of the RNN. In contrast to e-prop, this approach suggests to draw new random broadcast weights for each layer of the unrolled network, i.e., for each time step of the RNN. Figure 7c shows that this variation of random e-prop performs much worse. However an intermediate version where the random broadcast weights are redrawn every 20 ms performs about equally well as random e-prop for the considered task.

Replacing the eligibility trace by the corresponding term of the Clopath rule

The dependence of the synaptic plasticity rules from e-prop on the postsynaptic membrane potential through the pseudo-derivative in the eligibility traces yields some similarity to some previously proposed rules for synaptic plasticity, such as that of [5], which were motivated by experimental data on the dependence of synaptic plasticity on the postsynaptic membrane potential. We therefore tested the performance of random e-prop, where the eligibility trace was replaced by the corresponding term from the “Clopath rule”:

$$[v_j^t - v_{\text{th}}^+]^+ [\bar{v}_j^t - v_{\text{th}}^-]^+ \bar{z}_i^{t-1}, \quad (49)$$

where $\bar{v}_j^t$ is an exponential trace of the post synaptic membrane potential, with a time constant of 10 ms chosen to match their data. $[\cdot]^+$ is the rectified linear function. The thresholds v_{th}^- and v_{th}^+ were $\frac{v_{\text{th}}}{4}$ and 0 respectively. Supplementary Figure 7b shows that the resulting synaptic plasticity rule performed quite well.

Simulation details: pattern generation task

The performance in this task is reported as a normalized mean squared error (nmse) that we defined for this task as: $\text{nmse} = \frac{\sum_{t,k} (y_k^t - y_k^{*,t})^2}{\sum_{t,k} (y_k^{*,t} - \bar{y}_k^*)^2}$, where we set $\bar{y}_k^* = \frac{1}{T} \sum_t y_k^{*,t}$.

Details of the network model and of the input scheme: We used a network that consisted of 600 LIF neurons. Each neuron had a membrane time constant of $\tau_m = 20$ ms and a refractory period of 3 ms. The firing threshold was set to $v_{\text{th}} = 0.41$. Output neurons used a membrane time constant of $\tau_{\text{out}} = 20$ ms. The network received input from 20 input neurons, divided into 5 groups, which indicated the current phase of the target sequence similar to [1]. Neurons in group $i \in \{0, 4\}$ produced 100 Hz regular spike trains during the time interval $[200 \cdot i, 200 \cdot i + 200)$ ms and were silent at other times.

Details of the target pattern: The target signal had a duration of 1000 ms and each component was given by the sum of four sinusoids, with fixed frequencies of 1 Hz, 2 Hz, 3 Hz, and 5 Hz. At the start of learning, the amplitude and phase of each sinusoid in each component was drawn uniformly in the range $[0.5, 2]$ and $[0, 2\pi]$ respectively. This signal was not changed afterwards.

Details of the learning procedure: For learning, we computed gradients after every 1 second of simulation, and carried out the weight update using Adam [6] with a learning rate of $3 \cdot 10^{-3}$ and default hyperparameters. After every 100 iterations, the learning rate was decayed by a factor of 0.7. For random e-prop, the broadcast weights B_{jk} were sampled from a Gaussian distribution with a mean of 0 and a variance of $\frac{1}{n}$, where n is the number of network neurons.

Firing rate regularization, as described in Supplementary Note 2, was applied with $c_{\text{reg}} = 0.5$ and a target firing rate of $f^{\text{target}} = 10$ Hz.

Supplementary References

- [1] Nicola, W. & Clopath, C. Supervised learning in spiking neural networks with force training. *Nat. Comm.* (2017).
- [2] Graves, A. & Schmidhuber, J. Framewise phoneme classification with bidirectional lstm and other neural network architectures. *Neural Networks* (2005).
- [3] Graves, A., Mohamed, A.-R. & Hinton, G. Speech recognition with deep recurrent neural networks. *ICASSP* (2013).

- [4] Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R. & Schmidhuber, J. LSTM: A search space odyssey. *IEEE TNNLS* (2017).
- [5] Clopath, C., Büsing, L., Vasilaki, E. & Gerstner, W. Connectivity reflects coding: a model of voltage-based STDP with homeostasis. *Nat. Neuro.* (2010).
- [6] Kingma, D. P. & Ba, J. Adam: A method for stochastic optimization. *Preprint at <https://arxiv.org/abs/1412.6980>* (2014).
- [7] Bellec, G. *et al.* Biologically inspired alternatives to backpropagation through time for learning in recurrent neural nets. *Preprint at <http://arxiv.org/abs/1901.09049>* (2019).
- [8] Kappel, D., Legenstein, R., Habenschuss, S., Hsieh, M. & Maass, W. A dynamic connectome supports the emergence of stable computational function of neural circuits through reward-based learning. *eNeuro* (2018).
- [9] Bellec, G., Kappel, D., Maass, W. & Legenstein, R. Deep rewiring: Training very sparse deep networks. *ICLR* (2018).
- [10] Glass, J., Smith, A. & K. Halberstadt, A. Heterogeneous acoustic measurements and multiple classifiers for speech recognition (1999).
- [11] Amodei, D. *et al.* Deep speech 2 : End-to-end speech recognition in english and mandarin. *ICML* **48**, 173–182 (2016).
- [12] Graves, A., Fernández, S., Gomez, F. & Schmidhuber, J. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *ICML* 369–376 (2006).
- [13] Graves, A. Supervised sequence labelling. In *PhD Thesis from TU Munich* (2012).
- [14] Marszałek, M., Laptev, I. & Schmid, C. Actions in context. *CVPR* (2009).
- [15] Morcos, A. S. & Harvey, C. D. History-dependent variability in population dynamics during evidence accumulation in cortex. *Nat. Neuro.* (2016).
- [16] Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Computation* **9**, 1735–1780 (1997).
- [17] Sutton, R. S. & Barto, A. G. *Reinforcement Learning: An Introduction* (MIT press, 2018).
- [18] Bellemare, M. G., Naddaf, Y., Veness, J. & Bowling, M. The arcade learning environment: An evaluation platform for general agents. *JAIR*, DOI: [<http://doi.org/10.1613/jair.3912>] **47**, 253–279 (2013).
- [19] Mnih, V. *et al.* Asynchronous methods for deep reinforcement learning. In *ICML*, 1928–1937 (2016).
- [20] Lillicrap, T. P., Cownden, D., Tweed, D. B. & Akerman, C. J. Random synaptic feedback weights support error backpropagation for deep learning. *Nat. Comm.* (2016).
- [21] Nøkland, A. Direct feedback alignment provides learning in deep neural networks. In *NIPS* (2016).